



南開大學  
Nankai University

计算机学院  
计算机系统设计实验报告

PA5 实验报告

姓名：王一诺

学号：2312331

专业：计算机科学与技术

2026 年 6 月 4 日

# 目录

|                                 |           |
|---------------------------------|-----------|
| <b>1 PA5 实验概述</b>               | <b>2</b>  |
| <b>2 实现 binary scaling</b>      | <b>2</b>  |
| 2.1 实验要求                        | 2         |
| 2.2 整体思路                        | 2         |
| 2.3 修改的文件                       | 3         |
| 2.4 具体实现过程                      | 3         |
| 2.5 运行仙剑战斗                      | 9         |
| 2.6 思考题：比较 FLOAT 和 float        | 10        |
| 2.7 总结                          | 12        |
| <b>3 perf 性能数据分析与 NEMU 性能瓶颈</b> | <b>12</b> |
| 3.1 实验背景                        | 12        |
| 3.2 perf 输出摘要                   | 12        |
| 3.3 热点函数分组分析                    | 13        |
| 3.4 根据 perf 结果得到的结论             | 17        |
| 3.5 思考题：性能瓶颈的来源                 | 17        |
| 3.6 总结                          | 19        |
| <b>4 天下武功唯快不破</b>               | <b>19</b> |
| 4.1 章节主题                        | 19        |
| 4.2 解释执行为什么慢                    | 19        |
| 4.3 JIT 的核心思想                   | 20        |
| 4.4 cache 思想在 JIT 中的体现          | 20        |
| 4.5 为什么以基本块为翻译单位                | 21        |
| 4.6 RTL 在 JIT 中的作用              | 22        |
| 4.7 JIT 相比解释执行的优势               | 22        |
| 4.8 JIT 的复杂性                    | 23        |
| 4.9 理解和思考                       | 23        |
| 4.10 总结                         | 24        |

## 1 PA5 实验概述

本次 PA5 实验围绕程序运行支持与性能分析展开，主要包括以下三个部分：

- 实现 binary scaling，用整数定点数模拟仙剑奇侠传中的浮点数运算，使战斗流程能够正常运行。
- 使用 perf 采集和分析 NEMU 运行 nanos-lite 时的性能数据，定位 hot code，并结合 NEMU 设计分析性能瓶颈来源。
- 理解“天下武功唯快不破”章节中关于解释执行、JIT、translation block、cache 和 RTL 分层的思想，分析从设计层面提升模拟器性能的方向。

## 2 实现 binary scaling

### 2.1 实验要求

这是 PA5 实验中的第一个要求：实现 binary scaling，用整数模拟仙剑奇侠传中需要用到的浮点数操作。实现正确后，仙剑奇侠传可以在进入战斗时正常进行伤害计算、时间槽计算等原本依赖浮点数的逻辑。

实验指导中约定 FLOAT 使用 32 位整数表示一个实数，其中最高位是符号位，接下来的 15 位表示整数部分，最低 16 位表示小数部分。因此 FLOAT 本质上是一个 Q15.16 格式的定点数。若实数为  $a$ ，则其 FLOAT 表示大致为：

$$A = a * 2^{16}$$

由于这个映射把小数点固定在第 16 位之前，所以整数加减法可以直接用于 FLOAT 加减法；但乘法和除法会改变缩放倍数，需要额外调整。

### 2.2 整体思路

FLOAT 的核心思想是把实数映射为放大  $2^{16}$  倍后的整数。这样 NEMU 不需要支持 x87 浮点指令，也可以让仙剑奇侠传中的浮点相关逻辑继续运行。

本实验实现时采用以下原则：

- 使用 int 表示 FLOAT，保持原框架接口不变。
- 将小数位数统一定义为 16，即 `FLOAT_FBITS = 16`。
- 对 int 和 FLOAT 之间的转换只做整数移位或整数乘除。
- 对 FLOAT 乘法，在计算完两个定点数的乘积后，将结果缩小  $2^{16}$  倍。
- 对 FLOAT 除法，在做整数除法前，先将被除数放大  $2^{16}$  倍。
- 对 float 到 FLOAT 的转换，不直接对 float 做算术运算，而是读取 float 参数的 IEEE754 位模式，再用整数操作转换。
- 避免在 32 位 x86 目标中生成 NEMU 尚未实现的复杂指令，例如由 64 位移位生成的 `shrd/shld` 指令。因此最终实现中尽量使用 32 位整数移位、比较、加减和乘法。

## 2.3 修改的文件

本次为了完成 binary scaling，主要修改了以下两个文件：

### 2.3.1 navy-apps/apps/pal/include/FLOAT.h

该文件中实现了几个内联函数：

```

1 #define FLOAT_FBITS 16
2 #define FLOAT_SCALE (1 << FLOAT_FBITS)
3
4 static inline int F2int(FLOAT a) {
5     return a / FLOAT_SCALE;
6 }
7
8 static inline FLOAT int2F(int a) {
9     return a * FLOAT_SCALE;
10 }
11
12 static inline FLOAT F_mul_int(FLOAT a, int b) {
13     return a * b;
14 }
15
16 static inline FLOAT F_div_int(FLOAT a, int b) {
17     assert(b != 0);
18     return a / b;
19 }

```

### 2.3.2 navy-apps/apps/pal/src/FLOAT/FLOAT.c

该文件中实现了以下函数：

```

1 FLOAT f2F(float a);
2 FLOAT F_mul_F(FLOAT a, FLOAT b);
3 FLOAT F_div_F(FLOAT a, FLOAT b);
4 FLOAT Fabs(FLOAT a);

```

并额外实现了一些辅助函数，用来处理符号、绝对值、溢出检查和 48 位被除数除以 32 位除数的长除法。

## 2.4 具体实现过程

### 2.4.1 实现 FLOAT 与 int 的转换

由于 FLOAT 是把实数放大  $2^{16}$  倍后的整数，所以整数  $n$  转换成 FLOAT 时，只需要乘以  $2^{16}$ ；FLOAT 转换回整数时，只需要除以  $2^{16}$ 。

对应关系如下：

```

1 int2F(n) = n * 216
2 F2int(x) = x / 216

```

在 C 代码中， $2^{16}$  使用 `FLOAT_SCALE` 表示，因此实现为：

```
1 static inline int F2int(FLOAT a) {
2     return a / FLOAT_SCALE;
3 }
4
5 static inline FLOAT int2F(int a) {
6     return a * FLOAT_SCALE;
7 }
```

这里没有使用浮点运算，完全通过整数乘除实现。

#### 2.4.2 实现 FLOAT 与 int 的乘除

如果一个 FLOAT 数  $A$  表示实数  $a$ ，即  $A = a * 2^{16}$ ，那么  $A$  乘以整数  $b$  的结果为：

```
1 A * b = a * b * 2^16
```

这个结果仍然符合 FLOAT 的表示格式，所以可以直接做整数乘法。

除以整数  $b$  时同理：

```
1 A / b = a / b * 2^16
```

因此实现为：

```
1 static inline FLOAT F_mul_int(FLOAT a, int b) {
2     return a * b;
3 }
4
5 static inline FLOAT F_div_int(FLOAT a, int b) {
6     assert(b != 0);
7     return a / b;
8 }
```

#### 2.4.3 实现 FLOAT 乘以 FLOAT

设两个实数  $a$  和  $b$  的 FLOAT 表示分别为  $A$  和  $B$ ：

```
1 A = a * 2^16
2 B = b * 2^16
```

如果直接相乘：

```
1 A * B = a * b * 2^32
```

但目标结果应该是：

```
1 a * b * 2^16
```

因此理论上需要：

```
1 F_mul_F(A, B) = (A * B) / 2^16
```

最直接的写法是使用 64 位整数保存  $A * B$ ，再右移 16 位。但在实验环境中，32 位 GCC 会把 64 位右移编译成 shrd 指令。如果 NEMU 尚未实现 shrd，就会在运行仙剑战斗时出现 invalid opcode。因此最终实现没有直接使用 64 位右移，而是把两个 32 位定点数拆成整数部分和小数部分计算。

将  $x$  和  $y$  分解为：

```
1 x = x_hi * 2^16 + x_lo
2 y = y_hi * 2^16 + y_lo
```

则：

```
1 (x * y) / 2^16
2 = x_hi * y_hi * 2^16
3   + x_hi * y_lo
4   + y_hi * x_lo
5   + (x_lo * y_lo) / 2^16
```

对应实现如下：

```
1 FLOAT F_mul_F(FLOAT a, FLOAT b) {
2     uint32_t x = F_abs_u32(a);
3     uint32_t y = F_abs_u32(b);
4     uint32_t x_hi = x >> FLOAT_FBITS;
5     uint32_t x_lo = x & (FLOAT_SCALE - 1);
6     uint32_t y_hi = y >> FLOAT_FBITS;
7     uint32_t y_lo = y & (FLOAT_SCALE - 1);
8     uint32_t hi_product = x_hi * y_hi;
9     uint32_t mag = 0;
10
11     assert(hi_product <= 0xffffu);
12     mag = F_checked_add(mag, hi_product << FLOAT_FBITS);
13     mag = F_checked_add(mag, x_hi * y_lo);
14     mag = F_checked_add(mag, y_hi * x_lo);
15     mag = F_checked_add(mag, (x_lo * y_lo) >> FLOAT_FBITS);
16
17     return F_from_sign_mag(mag, (a < 0) ^ (b < 0));
18 }
```

其中 `F_abs_u32` 用于取出绝对值，`F_from_sign_mag` 用于根据符号恢复最终结果。这样乘法主体只处理非负数，符号单独处理，逻辑更清晰。

#### 2.4.4 实现 FLOAT 除以 FLOAT

设：

```
1 A = a * 2^16
2 B = b * 2^16
```

目标是得到：

```
1 (a / b) * 2^16
```

因为  $a = A / 2^{16}$ ,  $b = B / 2^{16}$ , 所以：

```
1 (a / b) * 2^16
2 = ((A / 2^16) / (B / 2^16)) * 2^16
3 = (A / B) * 2^16
4 = (A * 2^16) / B
```

因此理论公式为：

```
1 F_div_F(A, B) = (A * 2^16) / B
```

如果直接使用 64 位整数保存  $A \ll 16$ , 再除以  $B$ , 可能会引入 64 位除法运行时依赖或生成 NEMU 未支持的指令。为了避免这个问题, 实现中把被除数看作 48 位无符号数, 然后使用逐位长除法完成除法。

伪代码如下：

```
1 quotient = 0
2 remainder = 0
3 for bit from 47 downto 0:
4     remainder = remainder * 2 + dividend[bit]
5     if remainder >= divisor:
6         remainder = remainder - divisor
7         quotient[bit] = 1
8 return quotient
```

由于最终 FLOAT 结果仍然是 32 位, 因此如果商需要超过 32 位, 就说明结果溢出, 用 assert 捕捉。

核心代码如下：

```
1 static uint32_t F_div_u48_u32(uint32_t dividend_hi, uint32_t dividend_lo,
2     uint32_t divisor) {
3     uint32_t quotient = 0;
4     uint32_t remainder = 0;
5     int bit;
6
7     assert(divisor != 0);
8
9     for (bit = 47; bit >= 0; bit --) {
10         uint32_t next_bit;
11         uint32_t carry = remainder >> 31;
12
13         if (bit >= 32) {
14             next_bit = (dividend_hi >> (bit - 32)) & 1;
15         }
16         else {
17             next_bit = (dividend_lo >> bit) & 1;
```

```

18     }
19
20     remainder = (remainder << 1) | next_bit;
21     if (carry || remainder >= divisor) {
22         remainder -= divisor;
23         if (bit >= 32) {
24             assert(0);
25             return UINT32_MAX;
26         }
27         quotient |= 1u << bit;
28     }
29 }
30
31 assert(remainder < divisor);
32 return quotient;
33 }

```

F\_div\_F 调用该长除法函数：

```

1 FLOAT F_div_F(FLOAT a, FLOAT b) {
2     uint32_t dividend = F_abs_u32(a);
3     uint32_t divisor = F_abs_u32(b);
4     uint32_t mag = F_div_u48_u32(dividend >> FLOAT_FBITS,
5         dividend << FLOAT_FBITS, divisor);
6     return F_from_sign_mag(mag, (a < 0) ^ (b < 0));
7 }

```

这里  $\text{dividend} \gg 16$  是  $A * 2^{16}$  后的高 32 位， $\text{dividend} \ll 16$  是低 32 位。这样组合起来等价于构造 48 位被除数  $A \ll 16$ 。

#### 2.4.5 实现 float 到 FLOAT 的转换

实验指导要求  $\text{f2F}(\text{float } a)$  不能直接对 float 做算术运算，否则编译器可能生成 x87 浮点指令，这和当前实验阶段不希望依赖 x87 的目标冲突。

因此实现时直接读取 float 参数在内存中的 IEEE754 位模式：

```

1 uint32_t bits = *(uint32_t *)&a;

```

IEEE754 单精度浮点数由 1 位符号位、8 位指数、23 位尾数组成：

```

1 sign      = bits[31]
2 exponent  = bits[30:23]
3 fraction  = bits[22:0]

```

对于普通规格化浮点数，其真实值为：

```

1 (-1)^sign * (1.fraction) * 2^(exponent - 127)

```

将其转换成 Q15.16 定点数，需要再乘以  $2^{16}$ ：



```
1 FLOAT_value = mantissa * 2^(exponent - 127 - 23 + 16)
```

其中 mantissa 是 24 位整数形式的尾数：

```
1 mantissa = (1 << 23) | fraction
```

核心实现如下：

```
1 FLOAT f2F(float a) {
2     uint32_t bits = *(uint32_t *)&a;
3     uint32_t exp, frac;
4     uint32_t mantissa, value;
5     int shift;
6
7     exp = (bits >> 23) & 0xff;
8     frac = bits & 0x7fffff;
9
10    if (exp == 0) {
11        return 0;
12    }
13
14    assert(exp != 0xff);
15
16    mantissa = (1u << 23) | frac;
17    shift = (int)exp - 127 - 23 + FLOAT_FBITS;
18    if (shift >= 0) {
19        assert(shift < 32);
20        value = mantissa << shift;
21    }
22    else if (shift <= -32) {
23        value = 0;
24    }
25    else {
26        value = mantissa >> -shift;
27    }
28
29    if (bits >> 31) {
30        assert(value <= 0x80000000u);
31        return (FLOAT)(0u - value);
32    }
33    else {
34        assert(value <= 0x7fffffffu);
35        return (FLOAT)value;
36    }
37 }
```

如果 exponent 为 0，表示 0 或非规格化数。本实验中仙剑使用的常量不会依赖这些极小数，因此这里直接返回 0。若 exponent 为 255，表示 NaN 或无穷大，不属于本实验需要处理的正常数值，因此用 assert 捕捉。

### 2.4.6 实现 Fabs

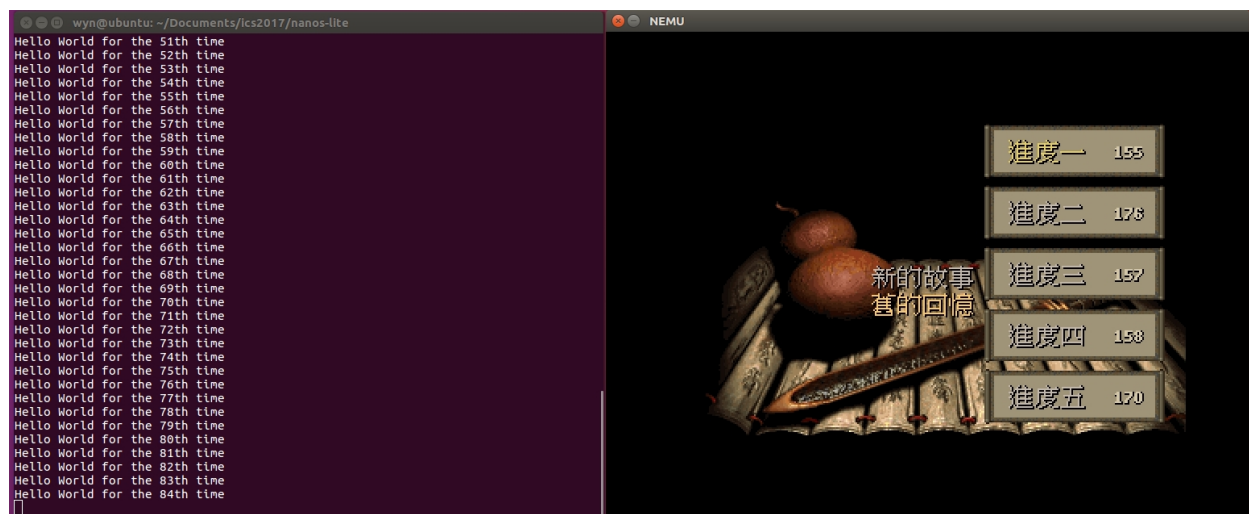
Fabs 需要返回 FLOAT 的绝对值。由于 FLOAT 使用补码表示负数，因此可以通过无符号运算安全取得幅值：

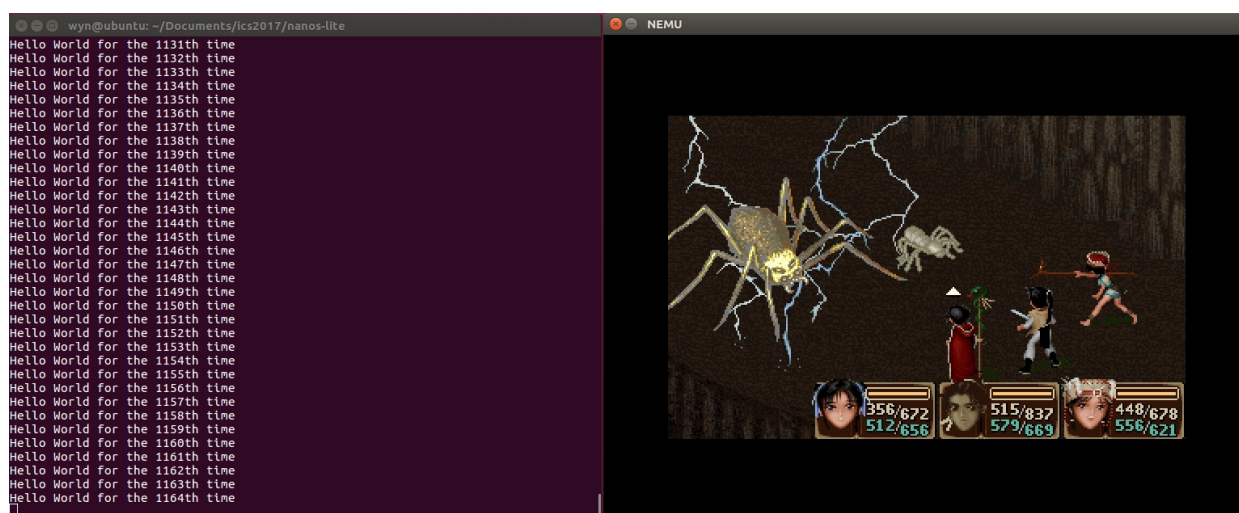
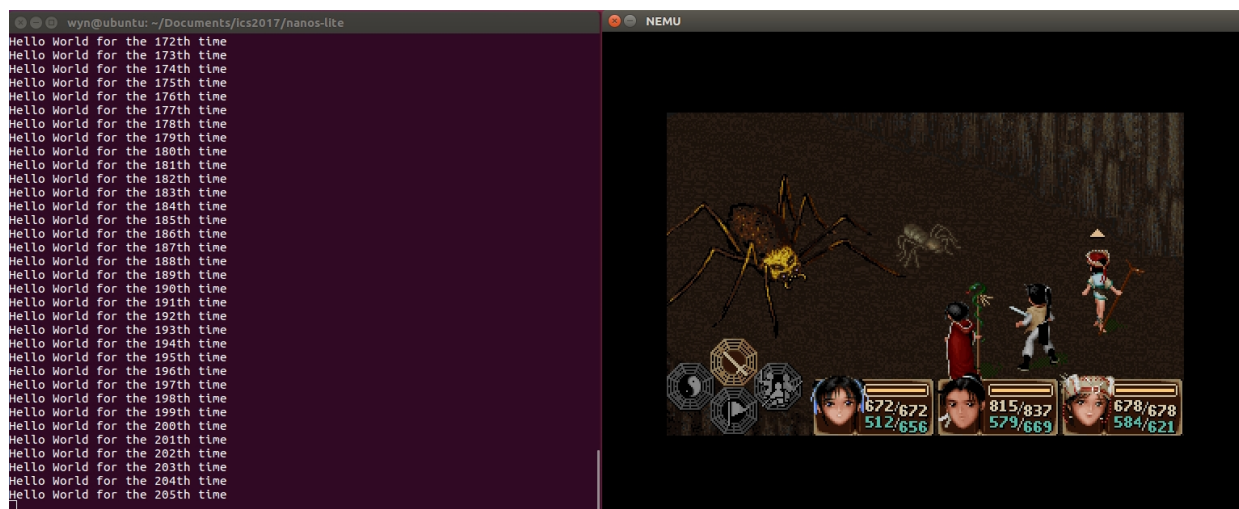
```
1 static uint32_t F_abs_u32(FLOAT a) {
2     return (a < 0) ? (0u - (uint32_t)a) : (uint32_t)a;
3 }
4
5 FLOAT Fabs(FLOAT a) {
6     uint32_t mag = F_abs_u32(a);
7     assert(mag <= 0x7fffffffu);
8     return (FLOAT)mag;
9 }
```

这里对最小负数的绝对值做了 assert 限制，因为 32 位有符号整数中 -2147483648 的正数形式无法用 int 表示。

## 2.5 运行仙剑战斗

实现正确后，就可以在仙剑奇侠传中成功进行战斗了：





## 2.6 思考题：比较 FLOAT 和 float

题目：FLOAT 和 float 类型的数据都是 32 位，它们都可以表示  $2^{32}$  个不同的数。但由于表示方法不一样，FLOAT 和 float 能表示的数集是不一样的。思考一下，我们用 FLOAT 来模拟表示 float，这其中隐含着哪些取舍？

回答如下。

### 2.6.1 表示范围的取舍

float 使用 IEEE754 单精度浮点格式。它的指数部分有 8 位，因此可以通过改变指数来表示非常大或非常小的数。float 的数量级范围大约可以覆盖  $10^{-38}$  到  $10^{38}$ 。

FLOAT 使用 Q15.16 定点格式，小数点位置固定。它没有独立的指数位，整数部分只有 15 位加符号位。因此它能表示的范围大致是：

$$-32768 \leq x < 32768$$

所以 FLOAT 的表示范围远小于 float。用 FLOAT 模拟 float 的第一个取舍就是牺牲动态范围。对于仙剑中的战斗数值、速度系数、伤害倍率等小范围实数，这个范围已经够用；但它不能通用地替代

float。

### 2.6.2 精度分布的取舍

float 的精度是相对精度。由于有指数位，float 在不同数量级上保持大致相同的有效数字位数。数值越大，两个相邻可表示数之间的绝对间隔也越大；数值越小，间隔也越小。

FLOAT 的精度是固定绝对精度。因为低 16 位表示小数，所以相邻两个 FLOAT 数之间的间隔恒定为：

```
1 1 / 216 = 0.0000152587890625
```

这意味着 FLOAT 在小范围普通数值上有稳定的小数精度，但对于特别大的数，精度不会随着数量级调整；对于特别小、低于  $1/2^{16}$  的数，则会直接舍入或截断为 0。因此，FLOAT 获得了固定、可预测的绝对精度，但失去了 float 的相对精度和对极小数的表达能力。

### 2.6.3 特殊值的取舍

float 可以表示一些特殊值，例如：

```
1 +0, -0, +inf, -inf, NaN, subnormal number
```

FLOAT 只是普通的 32 位补码整数定点数。它没有 NaN、无穷大、非规格化数等概念。除以 0、溢出等情况不能通过特殊浮点值自然表达，只能通过 assert 或其他错误处理方式捕捉。

因此，FLOAT 的语义更简单，但也失去了 IEEE754 的特殊值处理能力。

### 2.6.4 运算复杂度的取舍

float 的加减乘除由浮点硬件或浮点指令处理，硬件会自动完成对阶、规格化、舍入、异常处理等步骤。但在当前 NEMU 实验阶段，如果要支持这些指令，就要实现 x87 浮点架构细节，复杂度很高。

FLOAT 的加减法就是整数加减法，乘除法也可以用整数运算实现。这样绕过了 x87 指令实现，符合 KISS 原则，能让仙剑战斗逻辑先跑起来。

因此，FLOAT 降低了模拟器实现复杂度，但把一部分数值表达能力和 IEEE754 精确语义牺牲掉了。

### 2.6.5 舍入方式的取舍

float 运算通常遵循 IEEE754 舍入规则，例如默认的 round to nearest even。FLOAT 的转换和乘除实现更接近整数截断。例如 f2F 将 float 常量转换成定点数时，无法表示的小数部分会被截断；乘法中的低位也会因为右移而丢弃。

例如 1.2 不能被二进制定点数精确表示，其 FLOAT 表示实际接近：

```
1 1.19999695
```

因此，FLOAT 运算简单可控，但舍入误差模型和 float 不完全一致。

### 2.6.6 可移植性和指令依赖的取舍

使用 float 可能让编译器生成 x87 或 SSE 等浮点相关指令。在 NEMU 尚未支持这些指令时，程序会因为非法指令而无法运行。

使用 FLOAT 后，代码主要生成普通整数指令，对当前实验中的 NEMU 更友好。但实现时也要注意，某些 C 代码虽然看起来是整数运算，例如 64 位移位，也可能生成 NEMU 尚未实现的复杂整数指令。因此本实现进一步避免了 64 位移位和 64 位除法。

这说明，FLOAT 让程序摆脱浮点指令依赖，但要求实现者更关注编译后生成的整数指令是否被 NEMU 支持。

### 2.6.7 适用场景的取舍

FLOAT 适合数值范围有限、对绝对误差要求不高、希望避免浮点硬件或浮点指令的场景。仙剑奇侠传战斗中的倍率、时间槽和伤害计算正好属于这种情况。

float 更适合通用科学计算、大范围数值、需要 IEEE754 语义和特殊值处理的场景。

因此，用 FLOAT 模拟 float 的本质取舍是：为了降低 NEMU 实现难度和避免浮点指令，牺牲 float 的大动态范围、相对精度、特殊值语义和标准舍入行为；同时获得了实现简单、运行指令更基础、结果在固定范围内较可控的优点。

## 2.7 总结

本次实验通过 Q15.16 定点格式实现了仙剑奇侠传所需的 FLOAT 运算。实现中，整数转换和 FLOAT 与整数的乘除较为直接；FLOAT 与 FLOAT 的乘除需要特别处理缩放倍数；f2F 需要通过读取 IEEE754 位模式避免生成浮点指令。

在实现过程中还需要注意一点：虽然 binary scaling 的目标是避免浮点指令，但如果直接使用 64 位整数移位或除法，也可能生成当前 NEMU 未实现的复杂指令。因此最终实现使用 32 位分解乘法和逐位长除法，进一步降低了对特殊指令的依赖。

通过该实现，仙剑中的战斗计算可以在不实现 x87 浮点指令的前提下运行。

## 3 perf 性能数据分析与 NEMU 性能瓶颈

### 3.1 实验背景

该章节要求使用 GNU/Linux 的 perf 工具对 NEMU 运行 nanos-lite 时的性能进行采样分析。优化不能只凭感觉，需要先找到 hot code，也就是程序运行时间主要消耗的位置。perf record 负责采集运行时性能数据，perf report 负责查看 perf.data 中的采样统计结果。

本次使用的命令为：

```
1 sudo perf record nemu/build/nemu nanos-lite/build/nanos-lite-x86-nemu.bin
2 sudo perf report -f
```

运行后生成 perf.data，并通过 perf report 查看 NEMU 的热点函数。

### 3.2 perf 输出摘要

终端输出中的关键部分如下：

```

1 Samples: 36K of event 'cpu-clock', Event count (approx.): 9245750000
2 Overhead Command Shared Object Symbol
3 17.28% nemu nemu [...] page_translate.part.1
4 12.97% nemu nemu [...] vaddr_read
5 10.42% nemu nemu [...] is_mmio
6 9.66% nemu nemu [...] paddr_read
7 9.34% nemu nemu [...] read_ModR_M
8 6.51% nemu nemu [...] exec_real
9 4.37% nemu nemu [...] load_addr
10 2.62% nemu nemu [...] exec_wrapper
11 2.41% nemu nemu [...] operand_write
12 2.36% nemu nemu [...] device_update
13 2.26% nemu nemu [...] cpu_exec
14 2.18% nemu nemu [...] decode_mov_E2G
15 2.13% nemu nemu [...] exec_add
16 1.40% nemu nemu [...] decode_mov_G2E
17 1.09% nemu nemu [...] decode_G2E
18 0.99% nemu nemu [...] rtl_setcc
19 0.99% nemu nemu [...] exec_cmp
20 0.98% nemu nemu [...] decode_E2G
21 0.97% nemu nemu [...] exec_inc
22 0.90% nemu nemu [...] vaddr_write
23 0.73% nemu nemu [...] exec_2byte_esc
24 0.70% nemu nemu [...] exec_mov
25 0.62% nemu nemu [...] paddr_write
26 0.61% nemu nemu [...] decode_J
27 0.56% nemu libc-2.23.so [...] __memcpy_sse2_unaligned
28 0.46% nemu nemu [...] exec_jcc

```

从输出可以看出,采样事件为 cpu-clock,perf 共采集到约 36K 个样本,估计事件计数约为 9245750000。这里的 Overhead 是 flat profile,即样本直接落在某个函数内部的比例,不是包含子调用的 inclusive time。因此分析时不能简单认为某个函数独立造成了全部开销,而应结合 NEMU 的调用关系理解这些热点。

### 3.3 热点函数分组分析

#### 3.3.1 地址翻译和内存访问是最主要热点

perf 输出中排名最靠前的是:

```

1 17.28% page_translate.part.1
2 12.97% vaddr_read
3 10.42% is_mmio
4 9.66% paddr_read
5 0.90% vaddr_write
6 0.62% paddr_write

```

这些函数都属于 NEMU 的内存访问路径。NEMU 中客户程序每执行一条指令,通常至少需要读



取指令字节；如果该指令访问内存，还需要读取或写入操作数。启用分页后，虚拟地址访问需要先经过 `vaddr_read/vaddr_write`，再通过 `page_translate` 转换为物理地址，最后由 `paddr_read/paddr_write` 访问物理内存或 MMIO。

对应源码逻辑可以概括为：

```

1 uint32_t vaddr_read(vaddr_t addr, int len) {
2     if (!cpu.cr0.paging) {
3         return paddr_read(addr, len);
4     }
5
6     if ((addr & PAGE_MASK) + len <= PAGE_SIZE) {
7         return paddr_read(page_translate(addr, false), len);
8     }
9
10    for each byte:
11        data |= paddr_read(page_translate(addr + i, false), 1)
12 }

```

这说明每次虚拟地址读取都可能触发页翻译和物理地址读取。若访问跨页，还要拆成多个字节分别翻译和读取。虽然当前实现中有一个简单 TLB，但 `page_translate` 仍然出现在第一热点，说明地址翻译在整体运行中非常频繁。

`paddr_read` 中还会检查地址是否属于 MMIO：

```

1 uint32_t paddr_read(paddr_t addr, int len) {
2     int map_NO = is_mmio(addr);
3     if (map_NO != -1) {
4         return mmio_read(addr, len, map_NO);
5     }
6     return pmem_rw(addr, uint32_t) & mask;
7 }

```

而 `is_mmio` 的实现是线性扫描 MMIO 映射表：

```

1 int is_mmio(paddr_t addr) {
2     int i;
3     for (i = 0; i < nr_map; i++) {
4         if (addr >= maps[i].low && addr <= maps[i].high) {
5             return i;
6         }
7     }
8     return -1;
9 }

```

这解释了为什么 `is_mmio` 有 10.42% 的开销。即使绝大多数访问并不是 MMIO，NEMU 仍然在每次物理内存读写时进行一次 MMIO 区间检查。这种设计简单清晰，但在高频内存访问路径上会造成明显开销。

综合来看，内存路径相关函数的 flat overhead 已经超过 50%。这说明 NEMU 的大量时间并不是花在客户程序的某个业务逻辑上，而是花在模拟客户机内存系统的通用流程上：虚拟地址检查、页表

翻译、TLB 查询、物理地址访问、MMIO 判断等。

### 3.3.2 指令译码和 ModR/M 处理也是重要热点

perf 输出中另一个明显热点组是译码相关函数：

```
1 9.34% read_ModR_M
2 4.37% load_addr
3 2.18% decode_mov_E2G
4 1.40% decode_mov_G2E
5 1.09% decode_G2E
6 0.98% decode_E2G
7 0.61% decode_J
```

x86 指令格式复杂，很多指令需要 ModR/M 字节来描述寄存器操作数、内存操作数以及寻址方式。read\_ModR\_M 和 load\_addr 负责解析 ModR/M、SIB、位移等信息，并填充 Operand 结构。

其逻辑可以概括为：

```
1 读取 ModR/M 字节
2 解析 mod、reg、rm 字段
3 如果操作数在寄存器中：
4     设置寄存器操作数
5 否则：
6     根据 mod/rm/SIB/dispatch 计算有效地址
7     如果需要，读取内存操作数的值
```

这种译码流程每次执行客户指令时都会重新进行。即使同一条客户指令在循环中执行了上千次，NEMU 仍然会每次重新取指、重新解析 opcode、重新解析 ModR/M、重新计算寻址方式。这就是解释执行模拟器的典型开销。

在真机上，x86 前端也需要译码，但它有硬件译码器、微指令缓存、流水线、分支预测等机制。而 NEMU 使用 C 代码逐条解释，因此这些译码工作变成了普通 native 指令上的软件开销。

### 3.3.3 解释执行主循环本身也占用时间

perf 输出中与执行框架直接相关的函数包括：

```
1 6.51% exec_real
2 2.62% exec_wrapper
3 2.26% cpu_exec
```

cpu\_exec 中的主循环大致如下：

```
1 for (; n > 0; n --) {
2     exec_wrapper(print_flag);
3
4     if (nemu_state != NEMU_RUNNING) {
5         return;
6     }
7
8     device_update();
```



```

9
10 if (nemu_state != NEMU_RUNNING) {
11     return;
12 }
13 }

```

exec\_wrapper 每次只执行一条客户指令，内部完成：

```

1 decoding.seq_eip = cpu.eip;
2 exec_real(&decoding.seq_eip);
3 update_eip();
4 if (cpu.INTR && cpu.eflags.IF) {
5     raise_intr(TIMER_IRQ, cpu.eip);
6     update_eip();
7 }

```

exec\_real 做的事情是读取 opcode，设置操作数宽度，然后通过 opcode\_table 调用对应的 decode 和 execute 函数：

```

1 opcode = instr_fetch(eip, 1);
2 decoding.opcode = opcode;
3 set_width(opcode_table[opcode].width);
4 idex(eip, &opcode_table[opcode]);

```

这说明 NEMU 当前采用的是逐条解释执行模式。每执行一条客户指令，都要经历完整的取指、译码、执行、更新 eip、检查中断、设备更新等步骤。perf 中 exec\_real、exec\_wrapper、cpu\_exec 排在前面，正是这个设计的直接体现。

### 3.3.4 操作数写回和具体指令执行有一定开销

perf 输出中还出现了：

```

1 2.41% operand_write
2 2.13% exec_add
3 0.99% exec_cmp
4 0.99% rtl_setcc
5 0.97% exec_inc
6 0.70% exec_mov
7 0.46% exec_jcc

```

这些函数属于具体指令语义执行和操作数写回。以 operand\_write 为例，它需要根据 Operand 类型判断写入寄存器还是写入内存。exec\_add、exec\_cmp、exec\_inc 等则实现具体的 x86 指令语义，并维护 EFLAGS。

这里的热点说明客户程序中常见整数指令和条件判断很多，但这些单个指令执行函数的占比并不如地址翻译和译码高。也就是说，NEMU 并不是主要慢在 add/mov/cmp 这些语义本身，而是慢在围绕每条指令的模拟框架、地址访问和译码流程。

### 3.3.5 设备更新也有可见开销

perf 输出中:

```
1 2.36% device_update
```

cpu\_exec 在每条客户指令后都会调用 device\_update。设备更新用于处理定时器、键盘、VGA 等外设事件，让模拟器具备完整机器行为。这个设计简单直接，语义也容易理解：每条指令执行后都有机会推进外设状态。

但从性能角度看，若 device\_update 每条指令都被调用，它就会成为固定开销。客户程序执行的指令越多，这个固定开销越明显。实际机器中的外设不是由 CPU 每执行一条指令主动轮询一次，而是通过硬件时钟、中断和总线机制协作。

## 3.4 根据 perf 结果得到的结论

### 3.4.1 NEMU 的热点不集中在某一个业务函数

perf 输出中没有出现类似“某个应用函数占 80%”的情况，而是出现了 page\_translate、vaddr\_read、is\_mmio、read\_ModR\_M、exec\_real 等模拟器基础设施函数。这说明性能瓶颈来自模拟器本身的通用机制，而不是仙剑或 nanos-lite 中某个具体算法。

### 3.4.2 内存系统模拟是当前最明显的实现热点

地址翻译、虚拟地址访问、物理地址访问、MMIO 判断合计占比很高。这说明每条客户指令的取指和访存都需要经过多层软件模拟，内存路径的函数调用和检查成本非常高。

### 3.4.3 指令译码重复发生，导致解释执行开销明显

read\_ModR\_M、load\_addr 和多个 decode 函数占比较高，说明 x86 指令译码是高频工作。NEMU 没有缓存译码结果，因此循环中的同一条指令会被反复译码。

### 3.4.4 逐条解释执行是根本结构性瓶颈

exec\_real、exec\_wrapper、cpu\_exec 的占比说明，NEMU 每条客户指令都走一遍完整解释执行流程。这种设计非常适合教学和调试，但 native CPU 需要执行大量指令才能模拟一条客户指令。PA5 指导中提到的“用 n 条 native 指令模拟一条客户指令”正是 NEMU 性能不高的根本原因。

## 3.5 思考题：性能瓶颈的来源

题目：Profiler 可以找出实现过程中引入的性能问题，但却几乎无法找出由设计引入的性能问题。NEMU 毕竟是一个教学模拟器，当设计和性能有冲突时，为了达到教学目的，通常会偏向选择易于教学的设计。这意味着，如果不从设计上作改动，NEMU 的性能就无法突破上述取舍造成的障壁。纵观 NEMU 的设计，你能发现有哪些可能的性能瓶颈吗？

思考和回答如下。

### 3.5.1 为什么 profiler 难以直接指出“设计瓶颈”

perf 能告诉我们 `page_translate` 占 17.28%、`vaddr_read` 占 12.97%、`is_mmio` 占 10.42%，但它不会直接告诉我们“逐条解释执行设计导致重复取指译码”。这些百分比只是设计瓶颈在具体函数上的表现。

例如看到 `read_ModR_M` 占 9.34%，一种局部优化思路是把 `read_ModR_M` 写得更快。但即使它快一倍，只能降低一部分开销。真正的问题是：同一条指令在循环中被重复译码。如果不引入译码缓存或 JIT，重复译码仍然会发生。

同理，看到 `page_translate` 很热，可以优化 TLB 或减少函数调用；但只要每条客户内存访问都必须通过软件 MMU 路径，性能上限仍然受解释模拟设计限制。

因此 profiler 能指出“热点在哪里”，但“为什么这些热点不可避免地频繁出现”需要结合 NEMU 的整体设计来分析。

### 3.5.2 可能的优化方向

**局部优化方向** 根据 perf 输出，可以考虑以下局部优化：

- 1 优化 `is_mmio`，例如先快速判断普通内存范围，再扫描 MMIO 表
- 2 优化 `page_translate`，例如扩大或改进 TLB
- 3 减少 `vaddr_read/paddr_read` 的函数调用层数
- 4 对常见长度的内存访问做快速路径
- 5 减少每条指令后的 `device_update` 频率
- 6 对常见指令和常见寻址方式做专门快速路径

这些优化可以降低热点函数的常数开销，但仍属于解释器内部优化。

**设计级优化方向** PA5 指导中提出的更根本方向是 JIT。其基本思想是把客户指令翻译成 native 指令，并按基本块缓存：

```

1 while (1) {
2     tb = query_cache(cpu.eip);
3     if (cache_miss) {
4         tb = jit_translate(cpu.eip);
5         update_cache(cpu.eip, tb);
6     }
7
8     jump_to(tb);
9 }

```

这样可以减少重复译码、减少解释器分发，并让宿主机直接执行等价 native 指令。对于顺序执行的代码，可以一次翻译一个基本块，而不是每次只处理一条指令。

不过 JIT 实现复杂度很高，需要处理自修改代码、异常、中断、内存一致性、寄存器映射、基本块退出、代码缓存失效等问题。因此教学版 NEMU 选择解释执行，是为了让体系结构和指令执行过程更容易观察和理解。

### 3.6 总结

本次 perf 输出表明, NEMU 的主要热点集中在内存系统模拟、地址翻译、MMIO 判断、指令译码和逐条解释执行框架上。排名靠前的 `page_translate`、`vaddr_read`、`is_mmio`、`paddr_read`、`read_ModR_M`、`exec_real` 都不是客户程序中的业务逻辑, 而是 NEMU 为了模拟一台完整 x86 机器所必须执行的基础设施代码。

这说明 NEMU 的性能瓶颈并不是某个简单函数写得不够快, 而是解释执行模拟器的结构性开销: 每条客户指令都要由大量 native 指令完成取指、译码、访存、执行和状态更新。教学模拟器为了清晰性、可调试性和可扩展性, 使用了较多抽象层和逐步分解的函数结构, 这些设计让代码容易理解, 但也导致性能难以接近真实机器。

因此, perf 可以帮助定位 hot code; 而对 NEMU 性能上限的理解, 还必须结合其解释执行、软件 MMU、软件设备轮询、重复译码和抽象化指令语义等设计特点。若要突破这些设计带来的性能障碍, 需要从解释执行转向译码缓存、基本块缓存甚至 JIT 这样的设计级优化。

## 4 天下武功唯快不破

### 4.1 章节主题

“天下武功唯快不破”章节主要讨论 NEMU 为什么慢, 以及如果要从设计层面提升 NEMU 的性能, 可以采用怎样的思路。前一节通过 perf 找到了 NEMU 的热点函数, 但 perf 只能告诉我们时间花在哪里, 不能直接指出这些热点背后的根本设计原因。

这一节进一步指出, NEMU 性能低的根本原因不是某一两个函数写得不够快, 而是 NEMU 当前采用解释执行模式。解释执行意味着客户程序每执行一条指令, NEMU 都要用软件完整模拟这条指令的生命周期: 取指、译码、执行、更新 eip。真实机器中这些步骤由硬件电路高速完成, 而 NEMU 只能用宿主机上的 C 代码和 native 指令模拟。

因此, NEMU 的性能瓶颈本质上是:

```
1 一条客户指令 = 很多条 native 指令模拟完成
```

如果记一条客户指令平均需要  $n$  条 native 指令模拟, 那么 NEMU 的速度天然受  $n$  的大小限制。要让 NEMU 更快, 就要想办法减少这个  $n$ 。

### 4.2 解释执行为什么慢

在解释执行模式下, NEMU 的主循环大致可以理解为:

```
1 while running:
2     instr = fetch(cpu.eip)
3     decoded = decode(instr)
4     execute(decoded)
5     update_eip()
```

这个循环每次只处理一条客户指令。即使同一条指令在循环中执行了很多次, NEMU 仍然会重复进行取指和译码。

例如客户程序中有一条指令:

```
1 mov $0x1234, %eax
```

如果这条指令被执行 10000 次，解释执行模式就会重复 10000 次：

```
1 读取这条指令的字节
2 识别这是 mov 指令
3 解析立即数 0x1234
4 确认目标寄存器是 eax
5 把 0x1234 写入 cpu.eax
6 更新 eip
```

其中真正改变模拟机器状态的动作只有“把 0x1234 写入 cpu.eax”。取指、译码和计算下一条 eip 是为了弄清楚这条指令要做什么。但如果指令本身没有改变，那么每次执行时这些辅助步骤得到的结果都是一样的。

这说明解释执行中存在大量重复劳动。NEMU 慢，并不是因为某条指令的语义特别复杂，而是因为每条指令每次执行都要重新走完整解释流程。

### 4.3 JIT 的核心思想

JIT，即即时编译，其核心思想是：不要每次都解释同一条客户指令，而是在第一次遇到客户指令时，把它翻译成等价的 native 指令或 native 代码片段，之后再次执行到同一位置时，直接运行已经翻译好的 native 代码。

对于前面的例子：

```
1 客户指令: mov $0x1234, %eax
```

解释执行的做法是每次都分析它；JIT 的思路是第一次分析后生成等价的宿主机操作：

```
1 cpu.eax = 0x1234
```

或者更接近机器层面的理解：

```
1 mov $0x1234, (cpu.eax 对应的宿主机内存位置)
```

这样后续再执行到这条客户指令时，就不需要重新译码，而是直接执行已经生成好的 native 代码。因此 JIT 的关键收益是减少重复取指、重复译码和解释器分发开销。

### 4.4 cache 思想在 JIT 中的体现

JIT 并不是把整个客户程序一开始就全部翻译好，而是在程序运行过程中按需翻译。因为程序中有些代码可能永远不会被执行，提前翻译它们会浪费时间和空间。

因此 JIT 采用 cache 思想：用客户程序的 eip 查找之前是否已经翻译过对应代码。如果翻译过，就直接执行缓存中的翻译结果；如果没有翻译过，再进行翻译并放入缓存。

伪代码如下：

```
1 while running:
2     tb = query_cache(cpu.eip)
3     if tb does not exist:
```

```

4     tb = jit_translate(cpu.eip)
5     update_cache(cpu.eip, tb)
6     jump_to(tb)

```

这里 `tb` 是 translation block, 即翻译块。`query_cache` 根据当前客户机 `eip` 查询是否存在已翻译代码。`jit_translate` 负责把客户指令翻译成 native 代码。`update_cache` 把翻译结果保存起来。`jump_to` 则直接跳转到翻译后的 native 代码执行。

这和普通缓存的思想一致:

```

1 如果命中 cache:
2     直接使用已有结果
3 如果缺失 cache:
4     计算结果并存入 cache

```

在 NEMU 的场景中, “计算结果” 就是翻译客户指令, “已有结果” 就是已经生成好的 native 代码。

#### 4.5 为什么以基本块为翻译单位

章节中提出一个问题: 每次应该编译多少条客户指令?

最简单的做法是一次只翻译一条客户指令。但这样每执行完一条指令, 就要重新回到 JIT 框架中查询下一条指令是否已经被翻译。对于顺序执行的代码, 这种查询非常频繁, 而且大部分时候是冗余的。

例如一段连续指令:

```

1 instr1
2 instr2
3 instr3
4 instr4

```

如果 `instr1` 被执行了, 并且没有跳转, 那么 `instr2`、`instr3`、`instr4` 很可能会顺序执行。若一次只翻译一条指令, 就会反复进行:

```

1 查询 instr1 的翻译结果
2 执行 instr1
3 查询 instr2 的翻译结果
4 执行 instr2
5 查询 instr3 的翻译结果
6 执行 instr3
7 查询 instr4 的翻译结果
8 执行 instr4

```

这样虽然避免了重复译码每条指令, 但仍然保留了大量 cache 查询和跳转开销。

更合理的做法是一次翻译一个基本块。基本块是只有一个入口、一个出口的顺序执行指令序列, 中间没有会改变控制流的分支目标或跳转出口。以基本块为单位翻译后, 可以让宿主主机连续执行一段 native 代码, 直到遇到跳转、调用、返回、中断检查等需要离开基本块的位置。

基本块翻译模式可以概括为:

```

1 从当前 eip 开始:
2     读取并译码一条指令
3     将其翻译为 native 操作
4     如果该指令可能改变控制流:
5         结束当前基本块
6     否则:
7         继续翻译下一条指令

```

这样可以减少解释器循环和 `cache` 查询次数，提高执行连续代码的效率。

## 4.6 RTL 在 JIT 中的作用

章节中提到，x86 指令集非常复杂，如果直接考虑每条 x86 指令如何翻译到 `native` 指令，会非常麻烦。PA2 中引入的 RTL 可以缓解这个问题。

RTL 是一种中间表示。前端负责把客户指令翻译成 RTL，后端负责把 RTL 翻译成宿主机 `native` 指令。这样可以把复杂问题拆成两层：

```

1 客户指令 -> RTL -> native 指令

```

这种设计的优点是前端和后端解耦。如果要支持新的客户架构，只需要实现该架构到 RTL 的翻译；如果要支持新的宿主机架构，只需要实现 RTL 到该宿主机 `native` 指令的翻译。

可以表示为：

```

1 x86 客户指令    -> RTL -> x86 native
2 mips 客户指令   -> RTL -> arm native
3 riscv 客户指令  -> RTL -> x86 native

```

也就是说，RTL 使 NEMU 的 JIT 不必为每一对“客户架构和宿主架构”单独写一套翻译器，而是通过中间层组合。

这体现了编译器中常见的前端、中端、后端分层思想。前端处理源语言差异，后端处理目标机器差异，中间表示作为桥梁。

## 4.7 JIT 相比解释执行的优势

JIT 的主要优势可以概括为以下几点。

**减少重复译码** 解释执行中，循环里的同一条指令每次都要重新译码；JIT 翻译后，后续可以复用翻译结果。

**减少解释器分发** 解释执行每条指令都要查 `opcode_table`，并调用对应 `helper`。JIT 可以把多条指令合成一个连续 `native` 代码块，减少函数调用和间接分支。

**更接近真实硬件执行方式** 翻译后的代码直接修改 `cpu` 状态，避免每条指令都回到解释器框架。

**可以进一步优化** 有了基本块后，还可以做寄存器分配、常量传播、公共子表达式消除、去除冗余内存访问等优化。虽然 PA5 指导主要强调基本 JIT 思想，但这也说明 JIT 为后续优化打开了空间。



## 4.8 JIT 的复杂性

JIT 虽然性能潜力大，但实现难度也明显高于解释执行。解释执行的优点是每条指令的行为都在 C 函数里明确写出，调试时可以一步步观察 cpu 状态。而 JIT 需要动态生成代码、管理代码缓存，还要保证翻译后代码和解释执行语义完全一致。

可能遇到的问题包括：

**自修改代码** 如果客户程序修改了自己的代码区域，之前缓存的翻译块可能已经过期，需要失效或重新翻译。

**异常和中断** 解释执行可以每条指令后检查中断；JIT 执行基本块时，需要设计合适的退出点，保证中断和异常语义正确。

**基本块边界** 遇到跳转、调用、返回、条件分支、间接跳转时，如何结束基本块并正确衔接下一个块，需要仔细处理。

**状态同步** 翻译后的 native 代码可能把客户寄存器暂存在宿主寄存器中，退出基本块时必须保证 cpu 结构体中的状态正确。

**调试支持** NEMU 教学中常用单步执行、反汇编、断点、监视点等功能。JIT 后这些调试能力会变复杂，因为客户指令和 native 指令之间不再是一对一的简单关系。

因此，JIT 是典型的性能与实现复杂度之间的取舍。

## 4.9 理解和思考

我认为本章节的核心是引导我们从“优化某个函数”上升到“理解系统设计对性能上限的影响”。

perf 可以告诉我们 page\_translate、vaddr\_read、read\_ModR\_M 很热，但如果只盯着这些函数做局部优化，提升会有限。原因是这些热点不是偶然出现的，而是解释执行模式必然带来的结果。只要 NEMU 仍然每条指令都重新取指、译码、执行，热点就会以某种形式存在。

因此，“天下武功唯快不破”这一节实际说明了两个层次的优化：

- 实现层优化：例如优化 TLB、减少 MMIO 判断、减少函数调用。这些能降低常数开销。
- 设计层优化：例如引入 JIT、translation block、基本块缓存。这些改变执行模型，减少重复工作，提升性能上限。
- 前者适合局部改进，后者才可能突破解释执行模拟器的根本瓶颈。

前一节提到 Amdahl 定律，强调应该优化 hot code。本节进一步说明，如果 hot code 的来源是设计模式本身，那么只优化局部代码可能收益有限。

例如 perf 显示 read\_ModR\_M 很热，可以尝试把它写得更快。但只要解释执行仍然每次都调用 read\_ModR\_M，重复译码的总体开销仍然存在。JIT 的做法不是让每次译码更快，而是让同一段代码尽量只译码一次。

这可以理解为：

- |   |                  |
|---|------------------|
| 1 | 局部优化：减少一次译码的成本   |
| 2 | JIT 优化：减少译码发生的次数 |



后者改变了问题本身，因此更有可能带来数量级上的性能提升。

#### 4.10 总结

该章节从 NEMU 的低性能现象出发，指出解释执行模式的根本问题：客户程序的每条指令都要通过软件完成取指、译码、执行和状态更新，导致一条客户指令需要大量 native 指令模拟。

JIT 的核心思想是把客户指令翻译成等价 native 代码，并使用 cache 保存翻译结果，避免重复取指和译码。为了进一步减少查询和跳转开销，翻译单位不应局限于单条指令，而应扩展到基本块。RTL 则作为中间表示，把复杂的客户指令翻译问题拆成前端和后端两部分，提高可移植性和可维护性。

性能瓶颈有时并不是某行代码的问题，而是执行模型的问题。解释执行清晰、易调试、适合教学，但性能上限较低；JIT 复杂、难调试，但能通过缓存翻译结果和直接执行 native 代码显著降低重复开销。这正体现了系统设计中清晰性、正确性、可维护性和性能之间的权衡。